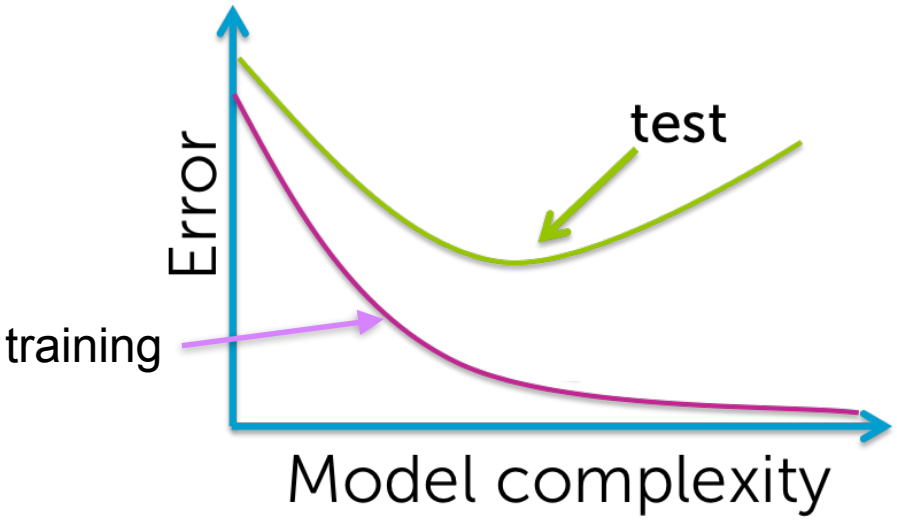
Lecture 7: Kernel Methods

Recap: Overfitting vs Underfitting

- General phenomenon

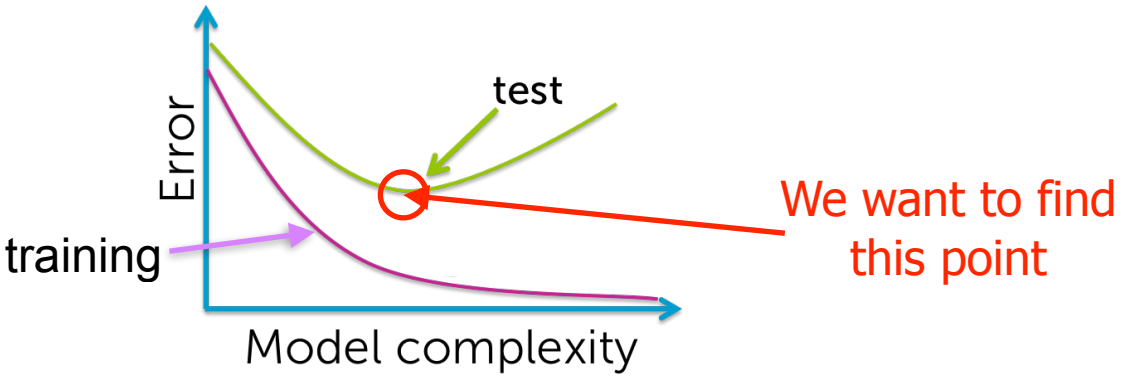


- Model complexity, e.g.
  - A high degree polynomial is more complex than a low degree one
  - In k-NN, $k = 1$ is more complex (the boundary is less smooth) than $k = 21$

Recap: Model selection

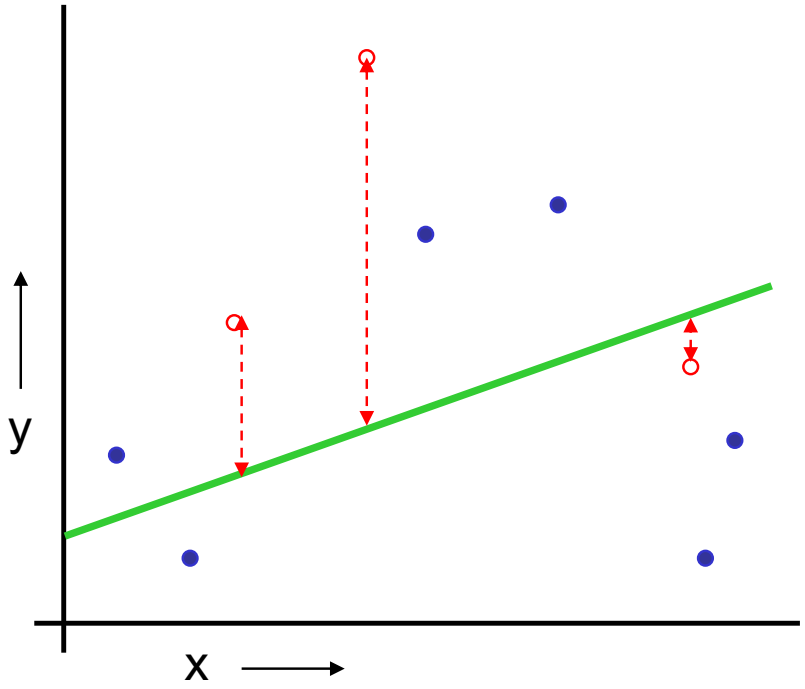
- The question then is:

Since we don't have access to the test data, how do we choose the right complexity for the model?



- Solution: Cross-validation
  - The following material was adapted from Andrew Moore's cross-validation slides

Recap: The validation set method

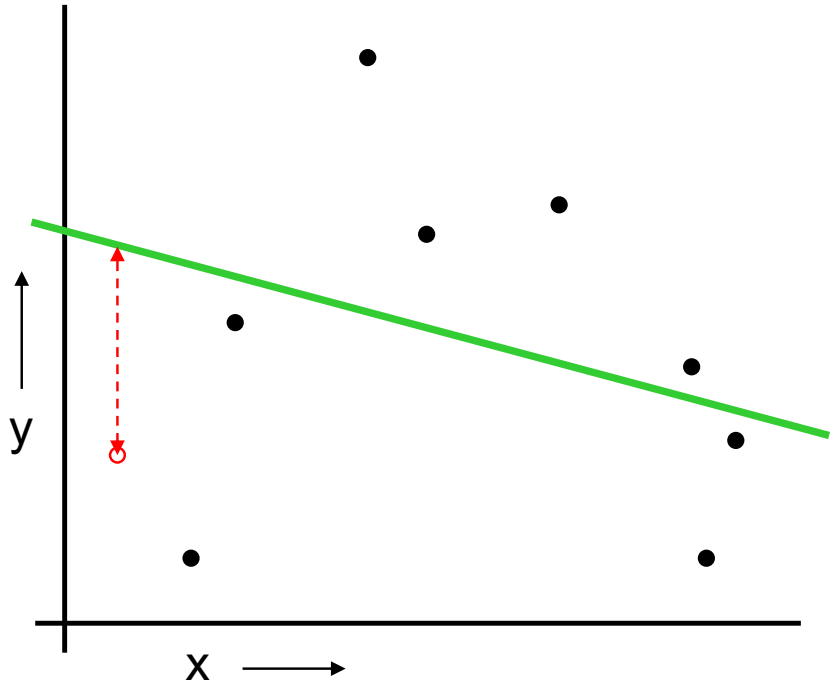


E.g., Fit a line

Mean squared error: 2.4

1. Randomly choose, e.g., 30% of the data to form a validation set
2. The remainder then acts as your new training set
3. Perform regression on this new training set only
4. Estimate the performance on the test data using the validation set

Recap: LOOCV



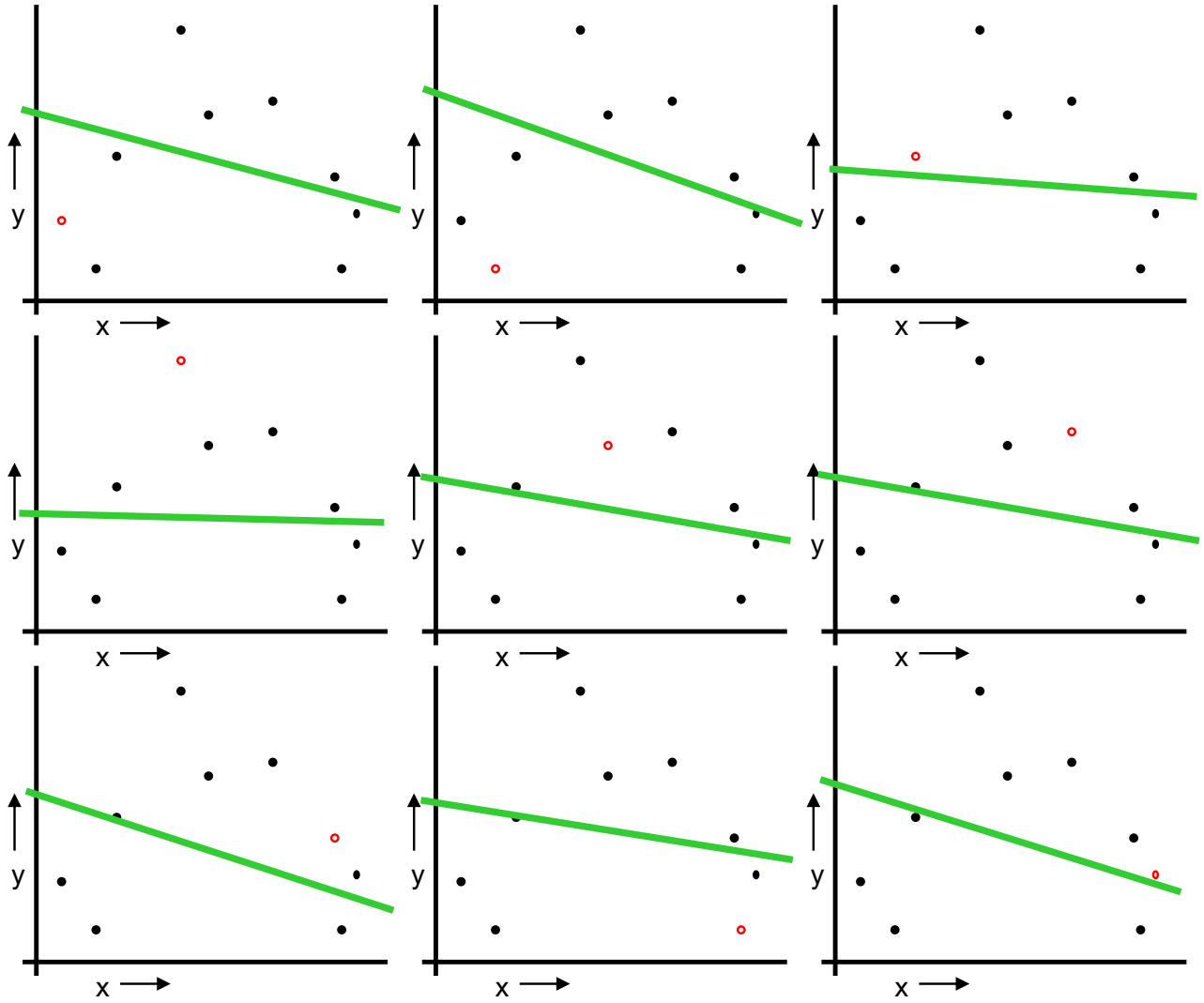
E.g., Fit a line

For $i = 1, \ldots, N$

1. Let (x_i, y_i) be the i^{th} sample
2. Temporarily remove (x_i, y_i) from the training set
3. Perform regression on the remaining $N - 1$ samples
4. Compute the error on (x_i, y_i)

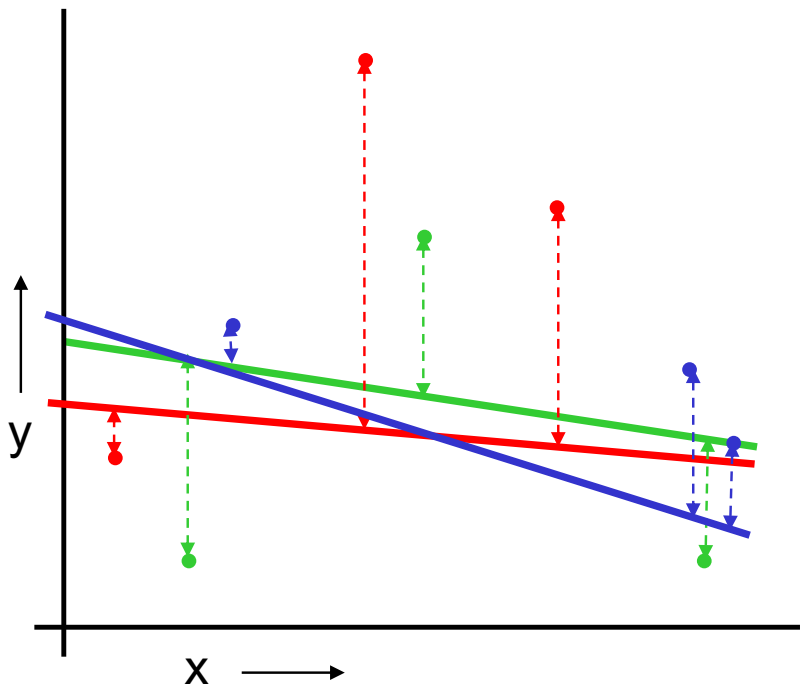
When you have done all points, report the average error

Recap: LOOCV



Mean squared error: 2.12

Recap: k-fold cross-validation



E.g., Fit a line

Mean squared error: 2.05

Randomly split the dataset into k partitions (e.g., $k = 3$, shown with different colors)

For the red partition, train on all the points **not** in the red partition. Compute the errors on the red points.

For the green partition, train on all the points **not** in the green partition. Compute the errors on the green points.

For the blue partition, train on all the points **not** in the blue partition. Compute the errors on the blue points.

Report the average error on all points

Recap: Which kind of cross-validation?

	Downside	Upside
Validation-set	Unreliable estimate of future performance	Cheap
Leave-one-out	Expensive	Doesn't waste data
10-fold	Wastes 10% of the data. 10 times more expensive than validation-set	Only wastes 10%. Only 10 times more expensive instead of N times
3-fold	Wastier than 10-fold. Expensivier than validation-set	Slightly better than validation-set
N-fold	Identical to Leave-one-out	

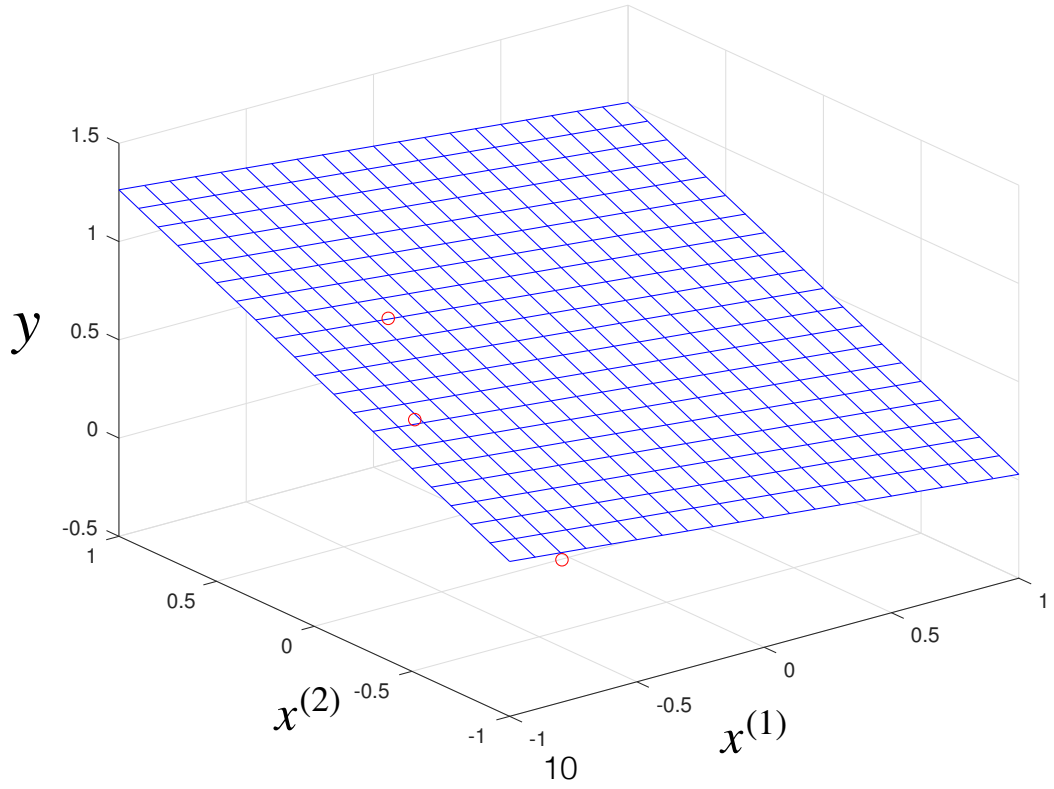
Recap: Overfitting with a linear model

- Because the linear model is simple, overfitting occurs fairly rarely

- Nevertheless, if there are fewer training samples than input dimensions ($N \leq D + 1$), then one can always fit a hyperplane that passes exactly through the training samples
  - This may seem a rare situation, but it can occur more easily when using feature expansion

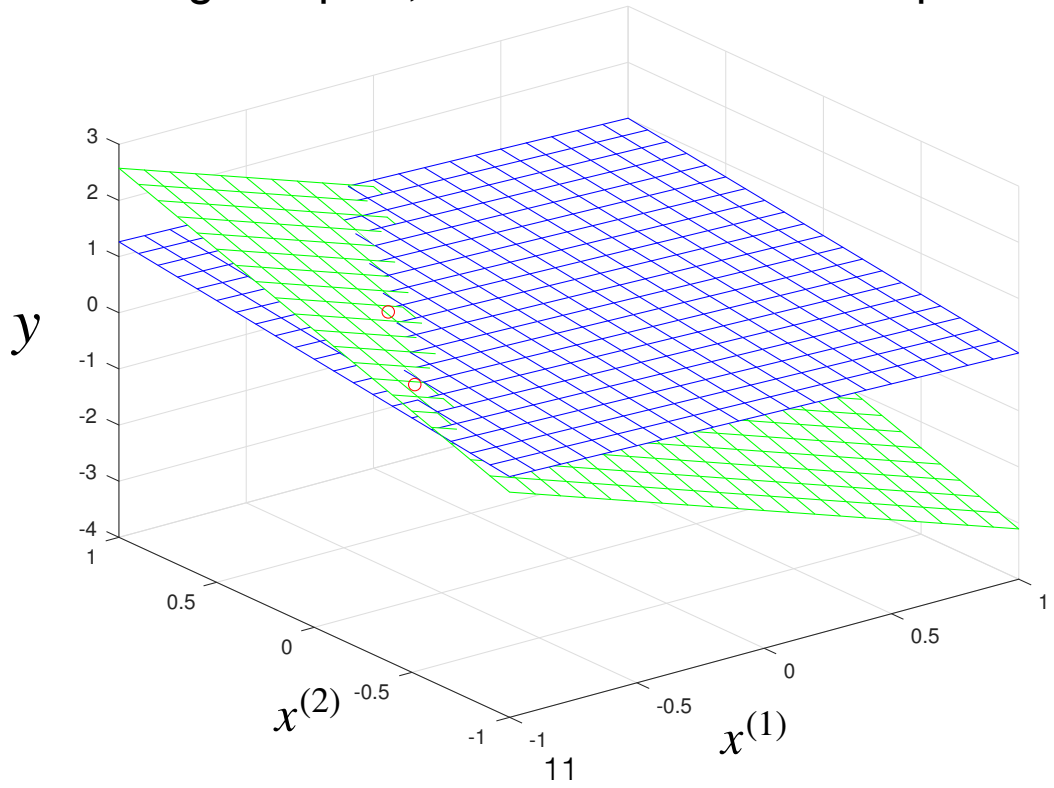
Overfitting with a linear model: Example

- Example: $N = 3,\ \ D = 2$
  - 3 noisy samples (red circles) originating from the true blue plane. They are not on the plane because of noise



Overfitting with a linear model: Example

- Example: $N = 3,\ \ D = 2$
  - 3 noisy samples (red circles) originating from the true blue plane. They are not on the plane because of noise
  - Fitting a plane to these samples yield the green plane. It passes exactly through the training samples, but is far from the true plane



Recap: Penalizing overfitting

- The standard way to penalize the complexity of a model is to add a regularizer to the empirical risk

- This leads to a new training objective function of the form

$$E(\mathbf{w}) = R(\mathbf{w}) + \lambda E_w(\mathbf{w})$$

where $R(\mathbf{w})$ is the empirical risk
$E_w(\mathbf{w})$ is the regularizer
λ is a hyper-parameter that defines the influence of the regularizer ($\lambda > 0$)

Recap: Regularized linear regression

- One of the most common regularizers is the sum of squares of the weights (there is a probabilistic motivation behind it)

$$E_w(\mathbf{w}) = \mathbf{w}^T\mathbf{w}$$

- This lets us write regularized linear regression as

$$\min_{\mathbf{w}} \sum_{i=1}^{N} (\phi(\mathbf{x}_i)^T\mathbf{w} - y_i)^2 + \lambda\mathbf{w}^T\mathbf{w}$$

- This remains a convex function (for $\lambda \geq 0$) and so we can still find the solution by zeroing out the gradient

Recap: Finding the right regularization strength

- Note that the use of a regularizer leads to another hyper-parameter to the model, i.e., λ
  - This parameter cannot be optimized with $\mathbf{w}$, because, for the training data, the best solution would always be $\lambda = 0$
  - Instead, it can be set via cross-validation, e.g., train with different values of $\lambda \in \{1e-3, 1e-2, 1e-1, 1, 10, 100, 1000\}$ and choose the best model based on the performance on validation data

Goals of today's lecture

- Introduce the notion of kernel function and the kernel trick

- Derive kernelized versions of linear (and ridge) regression, and look at examples of kernel SVM

Back to the linear model

- In dimension D, we can write

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + \ldots + w^{(D)}x^{(D)} = \mathbf{w}^T \begin{bmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{bmatrix}$$

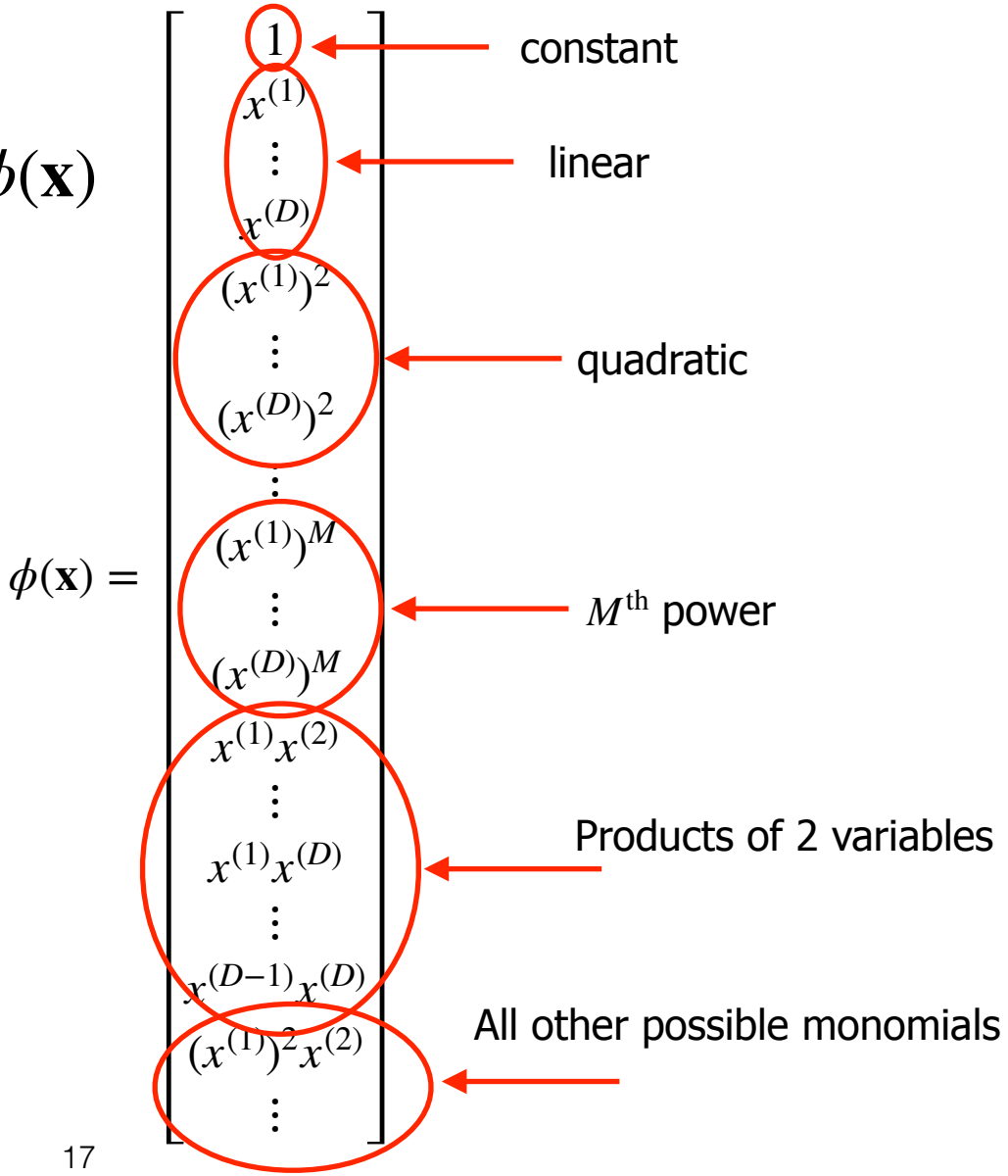
- Ultimately, whatever the dimension, we can write

$$y = \mathbf{w}^T\mathbf{x}$$

with $\mathbf{x} \in \mathbb{R}^{D+1}$, where the extra dimension contains a 1 to account for $w^{(0)}$

Recap: Polynomial feature expansion

- In general, for D-dimensional inputs, we can obtain a large $\phi(\mathbf{x})$
- The resulting expanded features can then be used in any linear algorithm
  - Let us look at how this would work for linear regression



Recap: Working with expanded features

- In essence, polynomial feature expansion uses a mapping from $\mathbf{x} \in \mathbb{R}^D$ to another representation $\phi(\mathbf{x}) \in \mathbb{R}^F$, where $F \gg D$

- We can then use a linear model in this new space and write

$$\hat{y}_i = \mathbf{w}^T \phi(\mathbf{x}_i)$$

where now $\mathbf{w} \in \mathbb{R}^F$

(assuming that the 1 accounting for the bias has been incorporated in $\phi(\mathbf{x}_i)$)

- Given training data, we can then find the optimal parameters via standard linear regression:

$$\min_{\mathbf{w}} \sum_{i=1}^{N} \left(\mathbf{w}^T \phi(\mathbf{x}_i) - y_i \right)^2$$

Recap: Working with expanded features: Solution

- We can group the transformed inputs $\{\phi(\mathbf{x}_i)\}$ and the outputs $\{y_i\}$ in a matrix and vector of the form

$$\Phi = \begin{bmatrix} \phi(\mathbf{x}_1)^T \\ \phi(\mathbf{x}_2)^T \\ \vdots \\ \phi(\mathbf{x}_N)^T \end{bmatrix} \in \mathbb{R}^{N \times F} \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in \mathbb{R}^N$$

- Then, we have

$$\mathbf{w}^* = \left(\Phi^T \Phi\right)^{-1} \Phi^T \mathbf{y}$$

Recap: Feature expansion: Properties

- Why limit ourselves to polynomials?
  - Any function of the input could be used, e.g.,
    - sine and cosine
    - exponential and logarithm
    - …
- In fact, in the demo…
  - https://playground.tensorflow.org/#activation=linear&batchSize=10&dataset=gauss&regDataset=reg-plane&learningRate=0.03&regularizationRate=0&noise=0&networkShape=&seed=0.49340&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false
- Then, how do we choose which functions to use?
  - Solution: Kernels: Let's circumvent this choice

Kernel function

- The solution to many ML algorithms ends up relying on a dot product between inputs, i.e., $\mathbf{x}_i^T \mathbf{x}_j$
- Because $\mathbf{x}_i^T \mathbf{x}_j \propto \cos\angle(\mathbf{x}_i, \mathbf{x}_j)$, this encodes a notion of similarity between two samples
- When doing feature expansion, we then have dot products of the form $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$, which also encodes a similarity between the two samples, but different from the original one
- Thus, instead of explicitly defining the mapping $\phi(\cdot)$, one can define a similarity function $k(\mathbf{x}_i, \mathbf{x}_j)$
- This is called a *kernel* function. It corresponds to some mapping $\phi(\cdot)$, i.e., $k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$, but this $\phi(\cdot)$ may be unknown

Kernel: Example

- One example of commonly-used kernel function is the polynomial kernel

$$k(\mathbf{x}_i, \mathbf{x}_j) = \left(\mathbf{x}_i^T \mathbf{x}_j + c\right)^d$$

- It has two hyper-parameters:
  - The bias c (often set to 1)
  - The degree of the polynomial d (often set to 2)

- For such kernels, the corresponding mappings $\phi(\cdot)$ are known
  - For example, with a 2D input, for $c = 1$ and $d = 2$, we have

$$\phi(\mathbf{x}) = \left[(x^{(1)})^2, (x^{(2)})^2, \sqrt{2}x^{(1)}x^{(2)}, \sqrt{2}x^{(1)}, \sqrt{2}x^{(2)}, 1\right]^T$$

Polynomial kernel vs feature expansion

- Consider the following mapping for a 2D input $\mathbf{x}$

$$\phi(\mathbf{x}) = \left[(x^{(1)})^2, (x^{(2)})^2, \sqrt{2}x^{(1)}x^{(2)}, \sqrt{2}x^{(1)}, \sqrt{2}x^{(2)}, 1\right]^T$$

- The dot product between two samples then is

$$\phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j) = (x_i^{(1)})^2(x_j^{(1)})^2 + (x_i^{(2)})^2(x_j^{(2)})^2 + 2x_i^{(1)}x_i^{(2)}x_j^{(1)}x_j^{(2)} + 2x_i^{(1)}x_j^{(1)} + 2x_i^{(2)}x_j^{(2)} + 1$$

$$= \left(x_i^{(1)}x_j^{(1)} + x_i^{(2)}x_j^{(2)} + 1\right)^2$$

- This corresponds to the quadratic kernel

$$k(\mathbf{x}_i, \mathbf{x}_j) = \left(\mathbf{x}_i^T\mathbf{x}_j + 1\right)^2$$

- Note that computing the kernel value only requires 3 multiplications, versus 6 when computing the dot product $\phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j)$

Kernel: Example

- Possibly the most common kernel is the Gaussian (or Radial Basis Function (RBF)) kernel

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

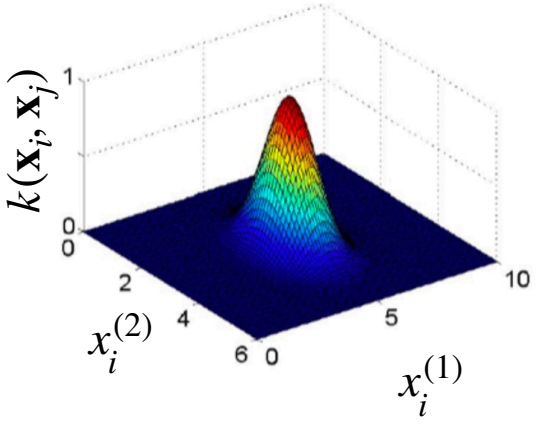
- It has one hyper-parameter:
  - The bandwidth σ (which is data-dependent)

- For this kernel, there is no explicit corresponding mapping $\phi(\cdot)$
  - In theory, this kernel corresponds to a mapping to an infinite dimensional space

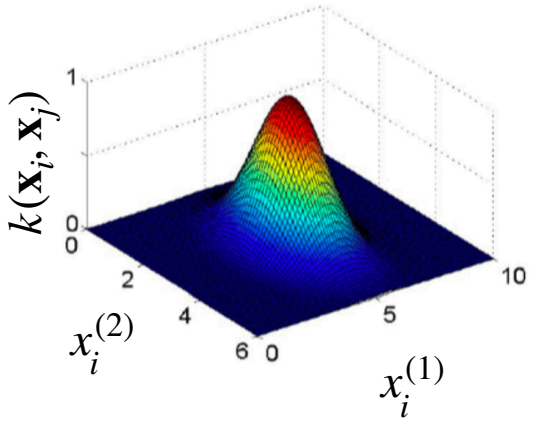
Gaussian kernel: Visualization

- With 2D inputs, we can visualize the effect of σ
- Let one sample be fixed as $\mathbf{x}_j = [5,3]^T$, and vary $\mathbf{x}_i$

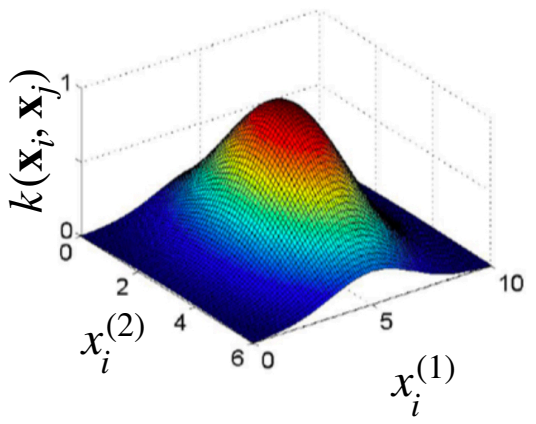
$\sigma^2 = 0.5$



$\sigma^2 = 1$



$\sigma^2 = 3$



Kernel trick

- Since a kernel function corresponds to a similarity, as a dot product, one can replace any dot product between two samples in the solution of an ML algorithm with a kernel function

- This is applicable to many algorithms, and is referred to as *kernelizing* an algorithm

- Let us now look at how this works for linear regression

Kernelization: Notation

- As mentioned before, the kernel function computes a similarity between two samples. It can be written as $k(\mathbf{x}_i, \mathbf{x}_j)$

- In what follows, with a small abuse of notation, I will also use the kernel function to encode the similarity between one sample and all the training samples, represented via the matrix $\mathbf{X}$. This will give

$$k(\mathbf{X}, \mathbf{x}_i) = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_i) \\ k(\mathbf{x}_2, \mathbf{x}_i) \\ \vdots \\ k(\mathbf{x}_N, \mathbf{x}_i) \end{bmatrix} \in \mathbb{R}^N$$

Kernelization: Notation

- Furthermore, I will also use the notion of kernel matrix, in which each entry (i, j) contains the kernel function evaluated between training sample i and training sample j. This matrix is squared and can be written as

$$\mathbf{K} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \cdots & k(\mathbf{x}_2, \mathbf{x}_N) \\ \vdots & \vdots & \vdots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & k(\mathbf{x}_N, \mathbf{x}_2) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix} \in \mathbb{R}^{N \times N}$$

Kernel regression

- As seen before, the solution to linear regression is given by

$$\mathbf{w}^* = \left(\Phi^T\Phi\right)^{-1}\Phi^T\mathbf{y} \qquad \Phi = \begin{bmatrix} \phi(\mathbf{x}_1)^T \\ \phi(\mathbf{x}_2)^T \\ \vdots \\ \phi(\mathbf{x}_N)^T \end{bmatrix} \in \mathbb{R}^{N\times F}$$

- At first sight, it may seem that $\Phi^T\Phi$ contains the dot products that we want to replace with a kernel function
  - However, $\Phi^T\Phi$ is an $F \times F$ matrix encoding dot products between the different feature dimensions, not the different samples
  - A kernel matrix, containing the dot products between every pair of training samples would be of size $N \times N$, and correspond to $\Phi\Phi^T$

Representer theorem

- To kernelize linear regression we can make use of the Representer theorem
  - This will be useful to kernelize other algorithms
- In essence, this theorem states that the minimizer of a regularized empirical risk function can be represented as a linear combination of the points in the high dimensional space. That is, we can write

$$\mathbf{w}^* = \sum_{i=1}^{N} \alpha_i^* \phi(\mathbf{x}_i) = \Phi^T \alpha^*$$

Proven by Schölkopf et al., “A Generalized Representer Theorem”, Computational Learning Theory, 2001

Kernel regression

- So instead of searching for a minimizer of the empirical risk in terms of $\mathbf{w}$, we search for one in terms of new variables $\{\alpha_i\}$
- That is, the empirical risk

$$R(\mathbf{w}) = \sum_{i=1}^{N} \left(\mathbf{w}^T \phi(\mathbf{x}_i) - y_i\right)^2$$

- is re-written as

$$R(\alpha) = \sum_{i=1}^{N} \left(\alpha^T \Phi \phi(\mathbf{x}_i) - y_i\right)^2$$

$$= \sum_{i=1}^{N} \left(\alpha^T k(\mathbf{X}, \mathbf{x}_i) - y_i\right)^2$$

Now, we have the kind of dot products we need

Vector obtained by evaluating the kernel function between all training samples and sample i

Kernel regression

- The gradient of the risk w.r.t. α is given by

$$\nabla R(\alpha) = 2 \sum_{i=1}^{N} k(\mathbf{X}, \mathbf{x}_i) \left(k(\mathbf{X}, \mathbf{x}_i)^T \alpha - y_i \right)$$

- Following the same reasoning as for linear regression, we can write this in matrix form as

$$\nabla_\alpha R(\alpha) = 2\mathbf{K}\mathbf{K}\alpha - 2\mathbf{K}\mathbf{y}$$

- $\mathbf{K}$ is the $N \times N$ training kernel matrix, obtained by evaluating the kernel function between all pairs of training samples

Kernel regression

- Setting the gradient to 0 yields

$$2\mathbf{K}\mathbf{K}\alpha^* = 2\mathbf{K}\mathbf{y}$$

$$\Leftrightarrow \mathbf{K}\alpha^* = \mathbf{y}$$

- which gives the solution

$$\alpha^* = (\mathbf{K})^{-1}\,\mathbf{y}$$

Kernel regression

- If we put this solution back in the definition of $\mathbf{w}^*$, we have

$$\mathbf{w}^* = \Phi^T \alpha^*$$

$$= \Phi^T (\mathbf{K})^{-1} \mathbf{y}$$

- This solution still depends on Φ, which we argued could be unknown

- In practice, this is not a problem, because our goal is to make predictions for new samples
  - We do not really care about having access to $\mathbf{w}^*$

Kernel regression: Prediction

- For a new sample $\mathbf{x}$, the prediction is given by

$$\begin{aligned}\hat{y} = (\mathbf{w}*)^T \phi(\mathbf{x}) &= \mathbf{y}^T (\mathbf{K})^{-1} \Phi\phi(\mathbf{x}) \\ &= \mathbf{y}^T (\mathbf{K})^{-1} k(\mathbf{X}, \mathbf{x})\end{aligned}$$

where $k(\mathbf{X}, \mathbf{x})$ is the N-dimensional vector obtained by evaluating the kernel function between the training samples and the test one

- Note that the quantity $\mathbf{y}^T (\mathbf{K})^{-1}$ only depends on the training data, and thus can be pre-computed (i.e., does not need to be re-computed for every test sample)

Exercise: Kernel regression

- You are given the following 4 training samples $(\mathbf{x}_i, y_i)$

$$\left(\begin{bmatrix}1\\1\end{bmatrix},0\right) \quad \left(\begin{bmatrix}0\\0\end{bmatrix},2\right) \quad \left(\begin{bmatrix}2\\0\end{bmatrix},2\right) \quad \left(\begin{bmatrix}2\\2\end{bmatrix},2\right)$$

- Compute the kernel matrix corresponding to a polynomial kernel of degree 2 with a bias of 1

Exercise: Kernel regression

- For the test sample $\mathbf{x}_t = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$, compute the entities needed for prediction (in addition to the kernel matrix)

Recap: Ridge regression

- To penalize overfitting, one can use a regularizer such as the sum of squares of the weights

$$E_w(\mathbf{w}) = \mathbf{w}^T\mathbf{w}$$

- Adding this to the linear regression empirical risk yields the ridge regression problem

$$\min_{\mathbf{w}} \sum_{i=1}^{N} (\phi(\mathbf{x}_i)^T\mathbf{w} - y_i)^2 + \lambda\mathbf{w}^T\mathbf{w}$$

- Such a regularizer can also be kernelized

Kernel ridge regression

- That is, with the Representer theorem, the regularized empirical risk

$$E(\mathbf{w}) = \sum_{i=1}^{N} \left(\mathbf{w}^T \phi(\mathbf{x}_i) - y_i\right)^2 + \lambda \mathbf{w}^T \mathbf{w}$$

- Can be re-written as

As the loss function, the regularizer has the dot products we need

$$E(\alpha) = \sum_{i=1}^{N} \left(\alpha^T \Phi \phi(\mathbf{x}_i) - y_i\right)^2 + \lambda \alpha^T \Phi \Phi^T \alpha$$

$$= \sum_{i=1}^{N} \left(\alpha^T k(\mathbf{X}, \mathbf{x}_i) - y_i\right)^2 + \lambda \alpha^T \mathbf{K} \alpha$$

Kernel ridge regression

- Following the same reasoning as without regularizer, we can write the gradient of the risk in matrix form as

$$\nabla_\alpha E(\alpha) = 2\mathbf{K}\left(\mathbf{K} + \lambda \mathbf{I}_N\right)\alpha - 2\mathbf{K}\mathbf{y}$$

- Setting it to 0 yields the solution

$$\alpha^* = \left(\mathbf{K} + \lambda \mathbf{I}_N\right)^{-1}\mathbf{y}$$

- Given α^*, one can then express $\mathbf{w}^*$ mathematically as in the case without regularization, and eventually use this formulation to make predictions for the test samples

Kernel regression: Demo

- Demo: http://www.tmpl.fi/gp/

- N.B. Strictly speaking this demo is for Gaussian Process (GP) regression; the mean prediction of a GP corresponds to the kernel regression prediction

Discussion

- As we just saw in the previous demo, the results of kernel ridge regression strongly depend on the choice of kernel and of the hyper-parameter values for a given kernel. How can you identify a good kernel and good hyper-parameter values?

Recap: Multi-output linear regression

- To predict multiple values, we use a parameter matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$, such that

$$\hat{\mathbf{y}}_i = \mathbf{W}^T \mathbf{x}_i = \begin{bmatrix} \mathbf{w}_{(1)}^T \\ \mathbf{w}_{(2)}^T \\ \vdots \\ \mathbf{w}_{(C)}^T \end{bmatrix} \mathbf{x}_i$$

where each $\mathbf{w}_{(j)}$ is a $(D+1)$-dimensional vector, used to predict one output dimension

Kernel ridge regression with multiple outputs

- To handle nonlinear problems, we can then again work in a different feature space $\phi(\cdot)$ and write the regularized empirical risk

$$E(\mathbf{W}) = \sum_{i=1}^{N} \|\mathbf{W}^T \phi(\mathbf{x}_i) - \mathbf{y}_i\|^2 + \lambda \|\mathbf{W}\|_F^2$$

- By making use of the Representer theorem, we can again derive a solution that does not explicitly depend on $\phi(\cdot)$ but rather on kernel values $k(\mathbf{x}_i, \mathbf{x}_j)$

Kernel ridge regression with multiple outputs

- Ultimately, the solution is given by

$$\mathbf{W}^* = \Phi^T (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{Y}$$

where Φ is the same matrix concatenating the training inputs $\{\phi(\mathbf{x}_i)\}$ as in the 1D-output case, and $\mathbf{Y} \in \mathbb{R}^{N \times C}$ is the matrix stacking the training output vectors

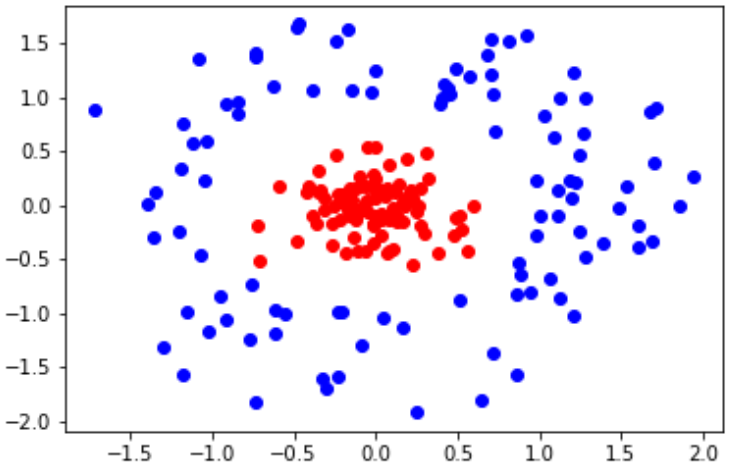
- The prediction vector for a test sample is then given as

$$\begin{aligned} \hat{\mathbf{y}} = (\mathbf{W}^*)^T \phi(\mathbf{x}) &= \mathbf{Y}^T (\mathbf{K} + \lambda \mathbf{I})^{-1} \Phi \phi(\mathbf{x}) \\ &= \mathbf{Y}^T (\mathbf{K} + \lambda \mathbf{I})^{-1} k(\mathbf{X}, \mathbf{x}) \end{aligned}$$

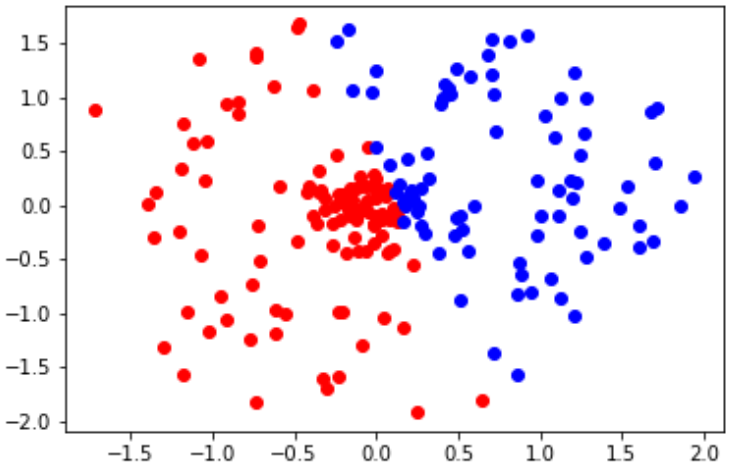
Kernel least-square classifier: Example 1

- 2D inputs from 2 classes (colors)

True labels



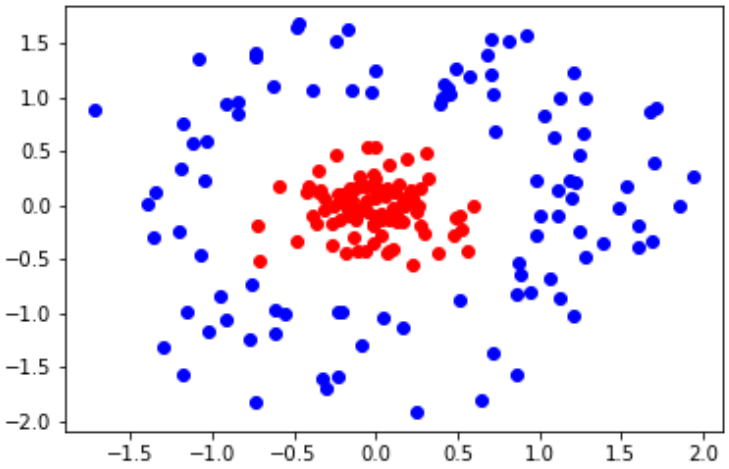
Fitting a linear model



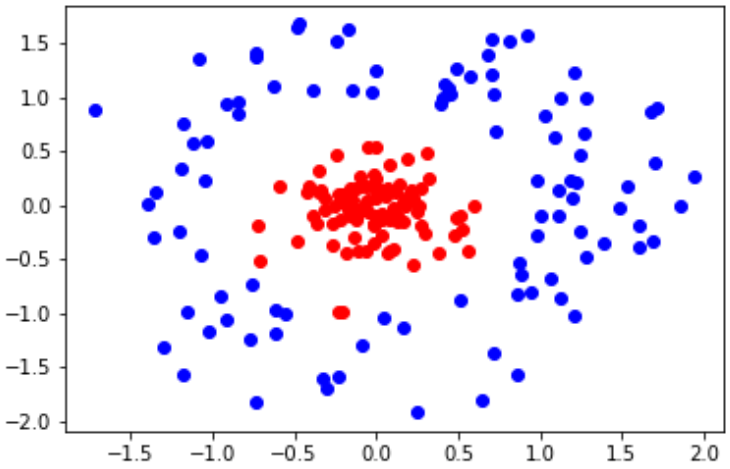
Kernel least-square classifier: Example 1

- 2D inputs from 2 classes (colors)

True labels



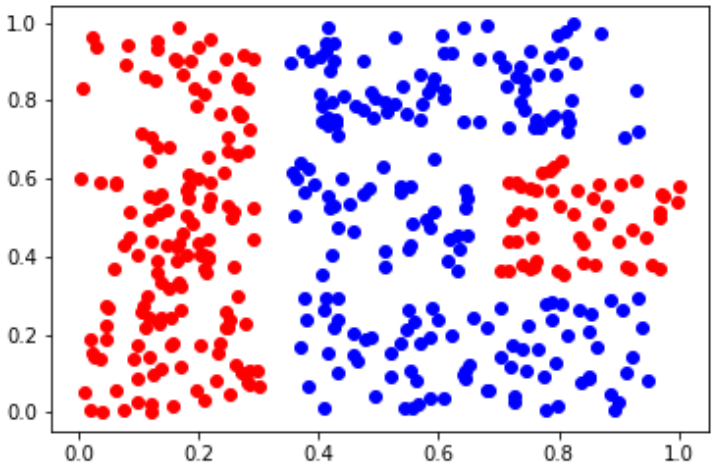
Fitting a nonlinear model
(RBF kernel)



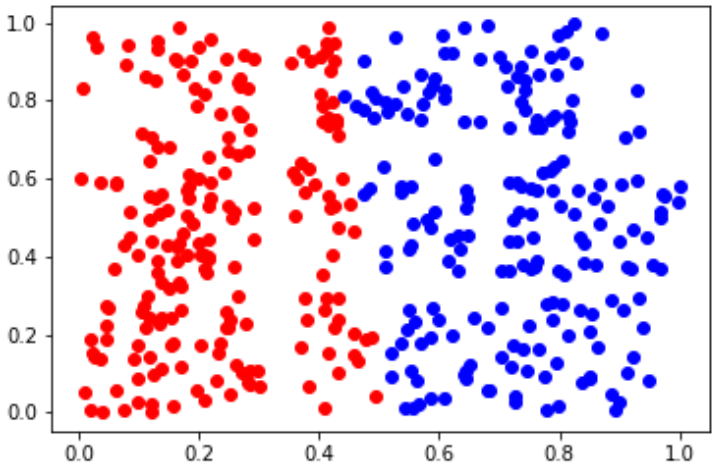
Kernel least-square classifier: Example 2

- 2D inputs from 2 classes (colors)

True labels



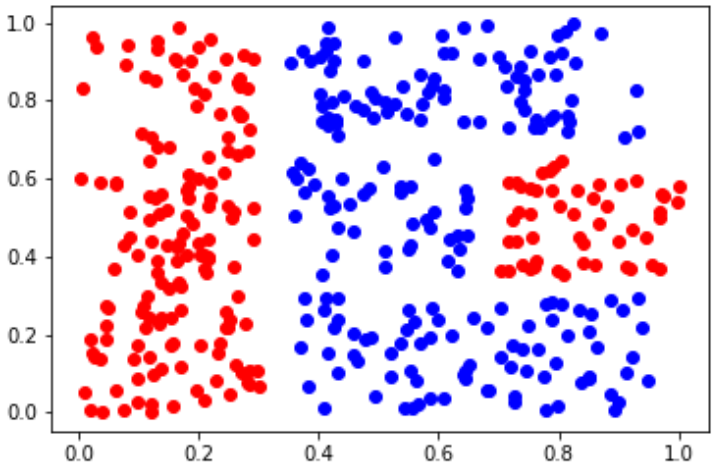
Fitting a linear model



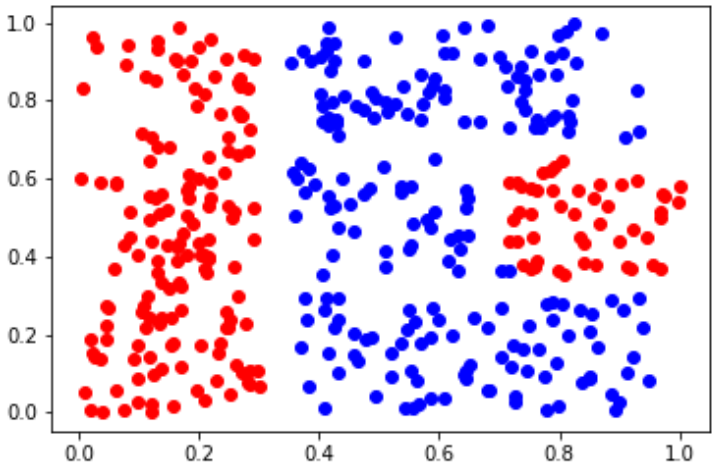
Kernel least-square classifier: Example 2

- 2D inputs from 2 classes (colors)

True labels



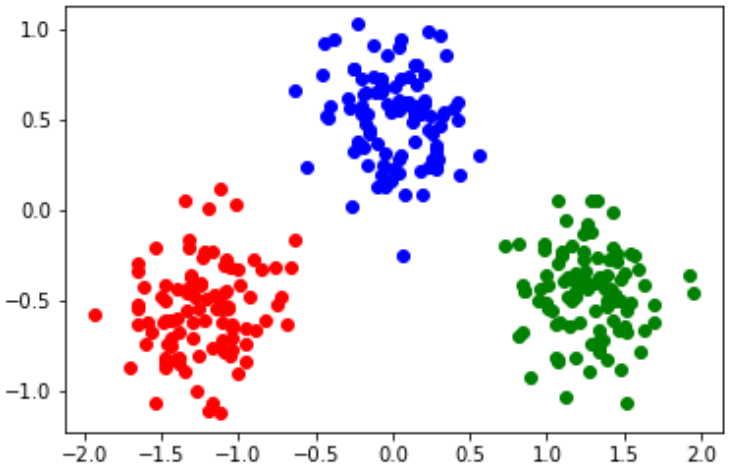
Fitting a nonlinear model
(RBF kernel)



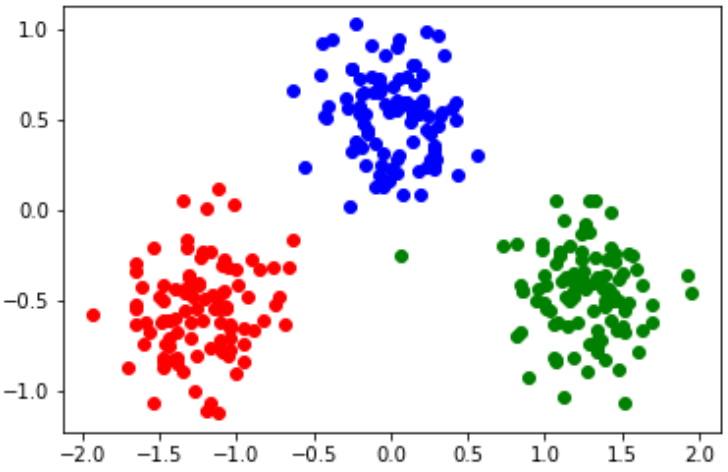
Kernel least-square classifier: Example 3

- The kernel least-square classifier also applies to linearly separable data
  - 2D inputs from 3 classes (colors)
  - One can use a linear kernel: $k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$

True labels



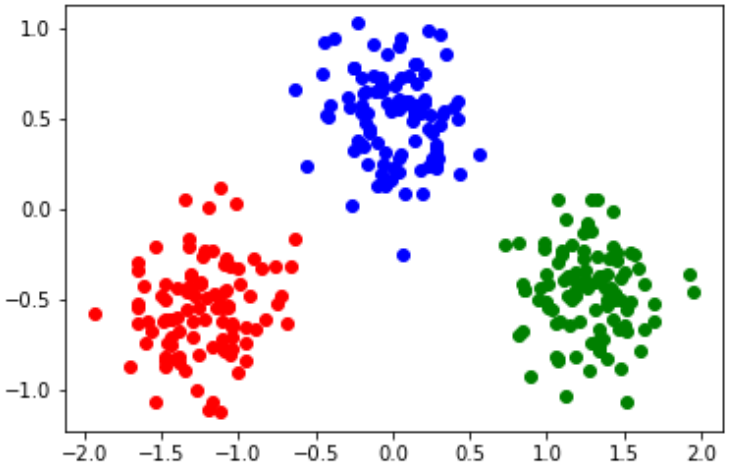
Fitted linear model
(KR with linear kernel)



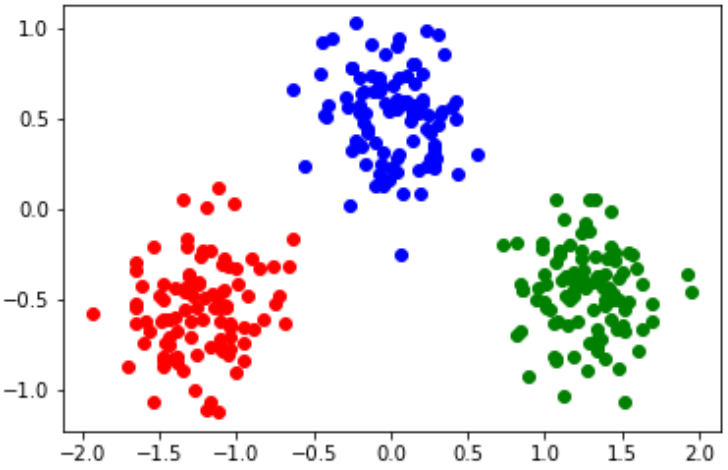
Kernel least-square classifier: Example 3

- With a regularizer, KRR also applies to linearly separable data
  - 2D inputs from 3 classes (colors)
  - Or an RBF kernel (here with a small λ)

True labels



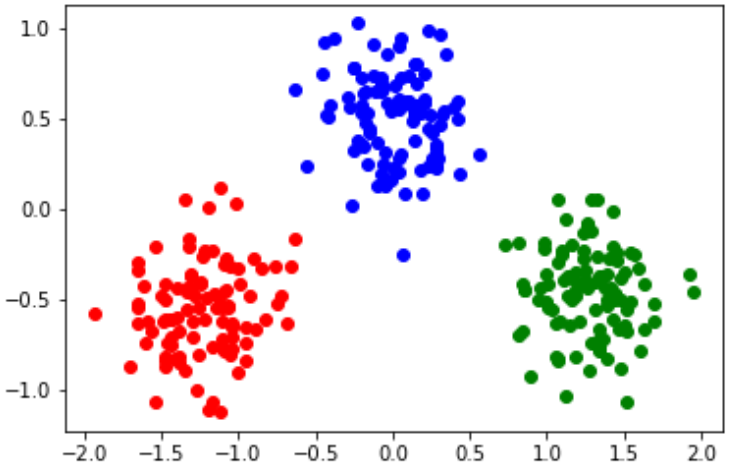
Fitted nonlinear model
(KRR with RBF kernel)



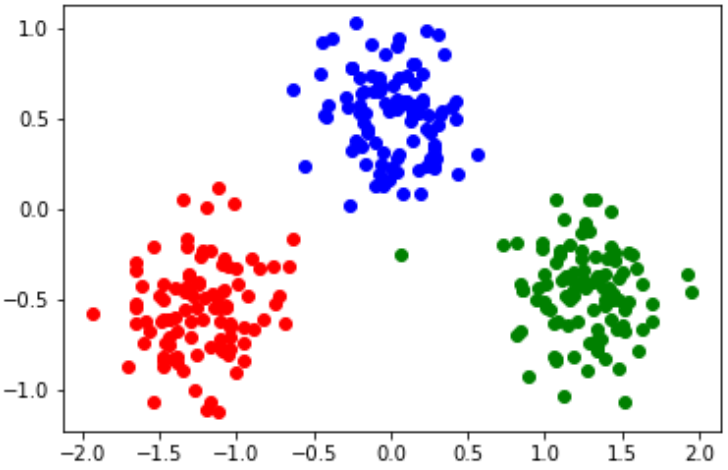
Kernel least-square classifier: Example 3

- With a regularizer, KRR also applies to linearly separable data
  - 2D inputs from 3 classes (colors)
  - Or an RBF kernel (here with a larger $\lambda = 1$)

True labels



Fitted nonlinear model
(KRR with RBF kernel)



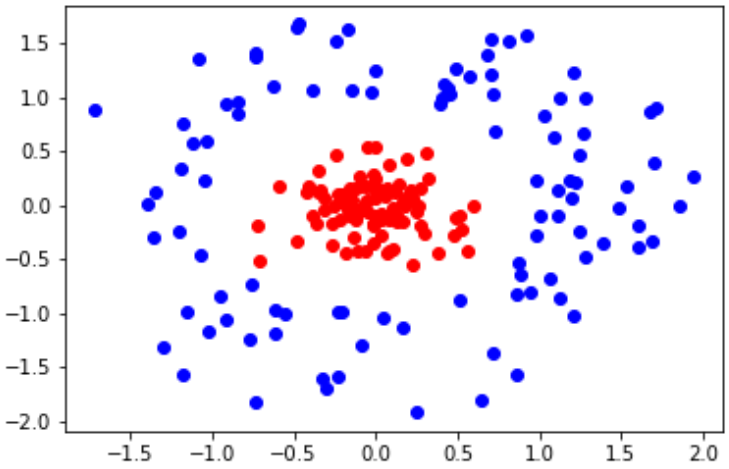
Kernelization

- The idea of kernelizing an algorithm does not apply only to linear regression
  - It can also be used to obtain a nonlinear version of SVM (via the so-called dual formulation of SVM)
  - In a few weeks, we will also do this for dimensionality reduction
  - Note that logistic regression can also be kernelized, although this is less commonly done

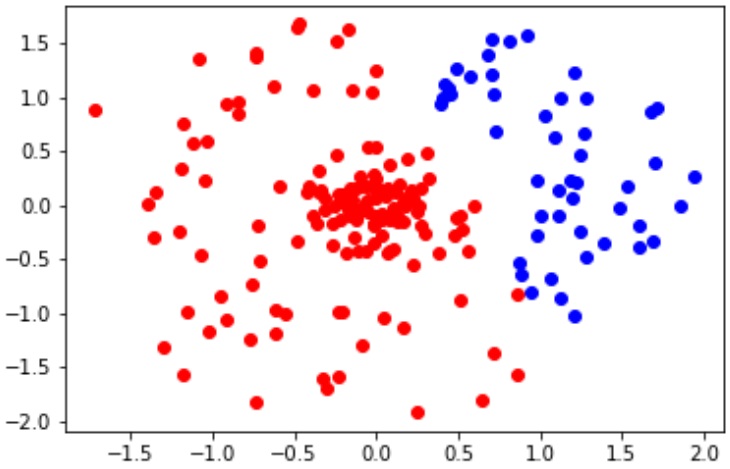
Kernel SVM: Example 1

- 2D inputs from 2 classes (colors)

True labels



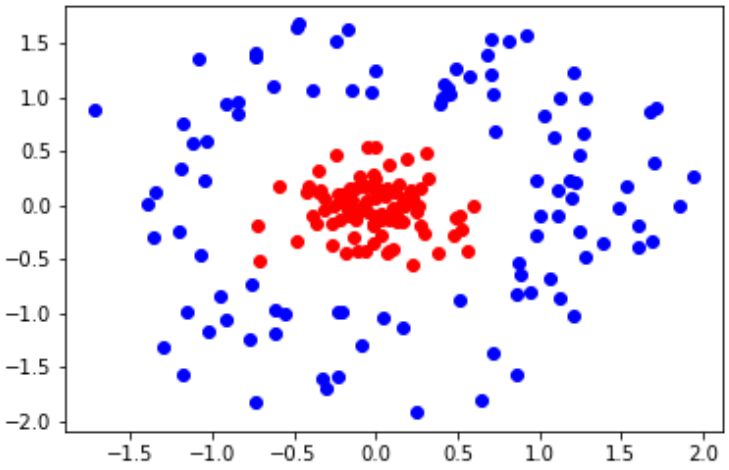
Fitting a linear SVM



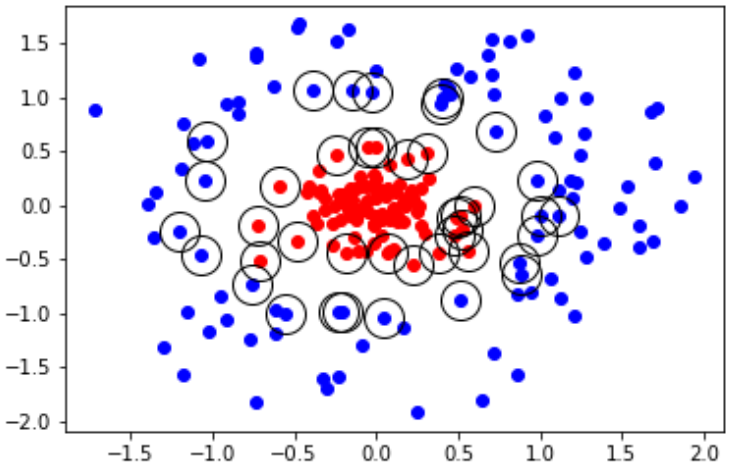
Kernel SVM: Example 1

- 2D inputs from 2 classes (colors)

True labels



Fitting a kernel SVM
(polynomial kernel)

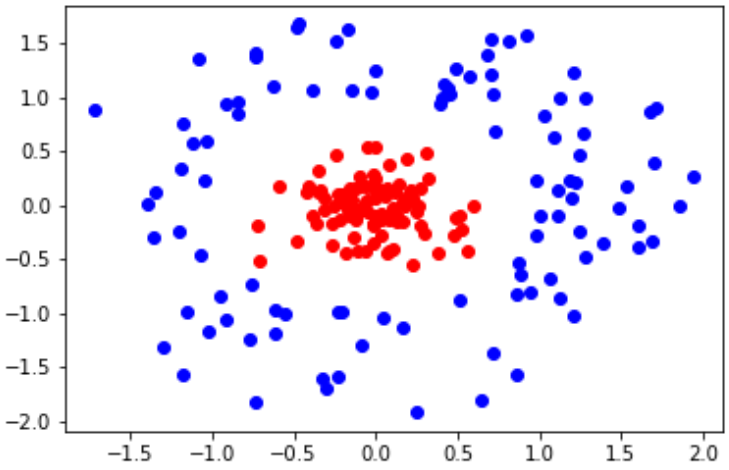


Circles indicate support vectors

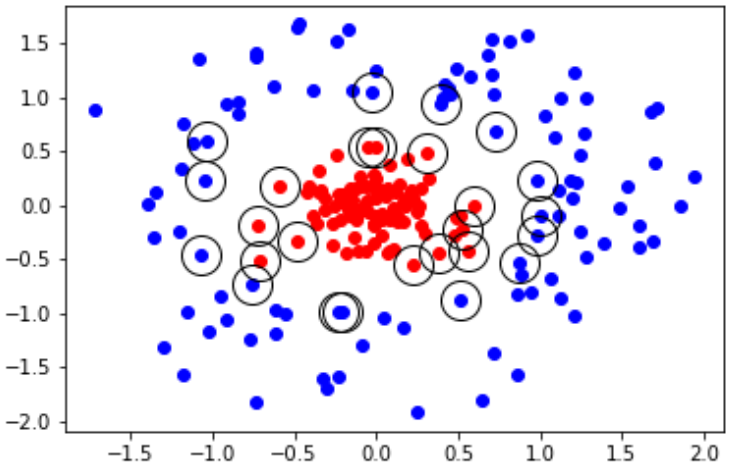
Kernel SVM: Example 1

- 2D inputs from 2 classes (colors)

True labels



Fitting a kernel SVM
(RBF kernel, $\sigma^2 = 1$)

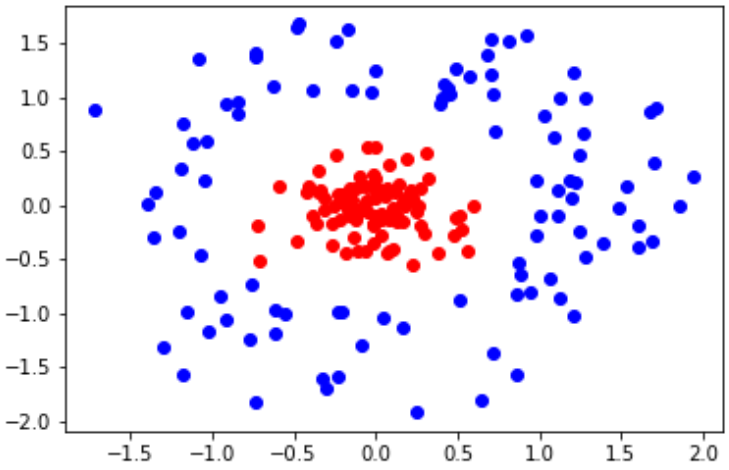


Circles indicate support vectors

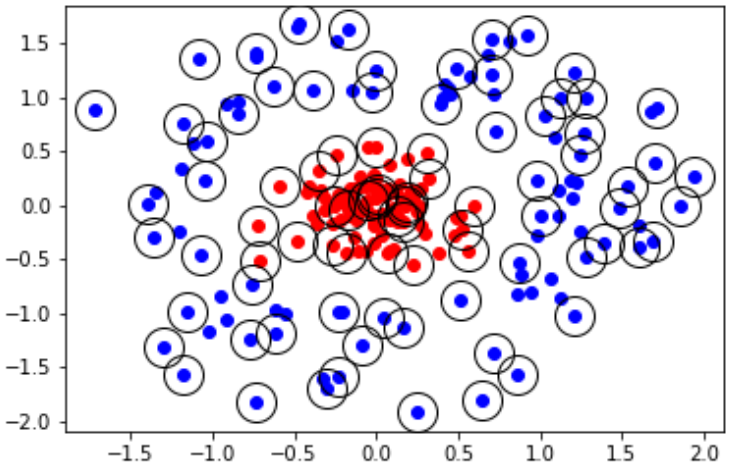
Kernel SVM: Example 1

- 2D inputs from 2 classes (colors)

True labels



Fitting a kernel SVM
(RBF kernel, $\sigma^2 = 10$)

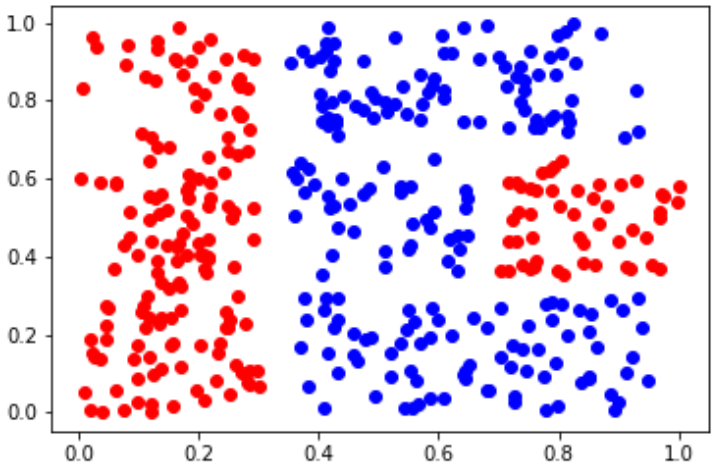


Circles indicate support vectors

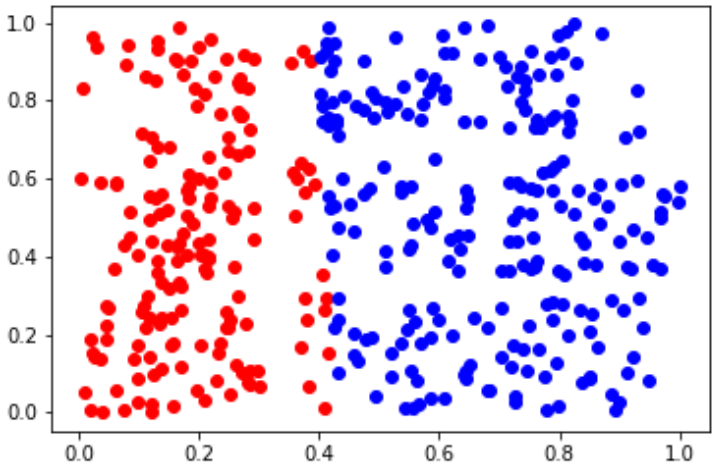
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



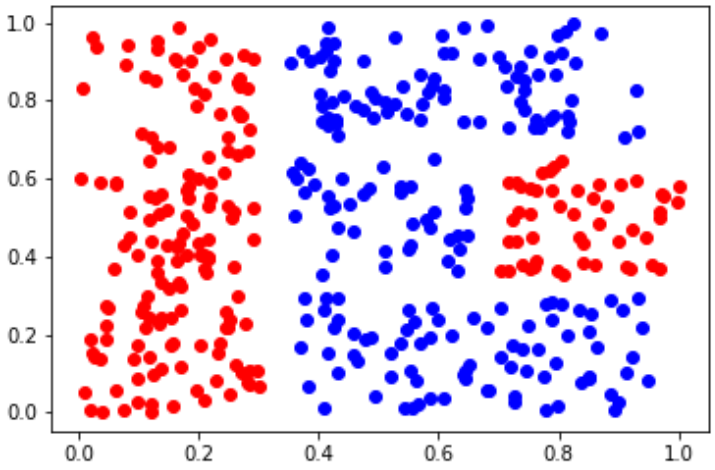
Fitting a linear SVM



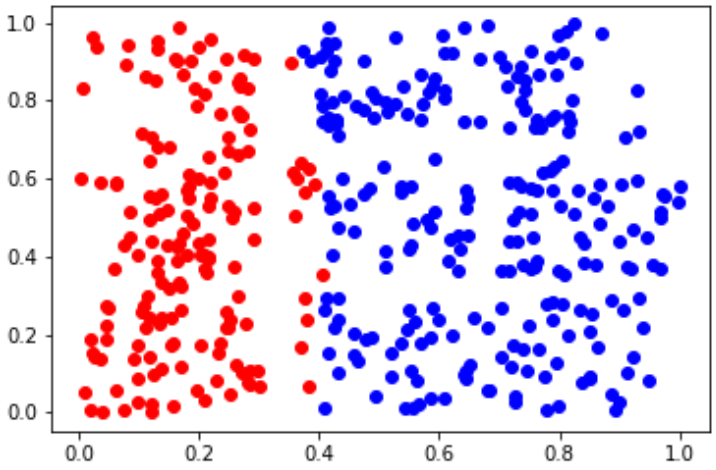
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



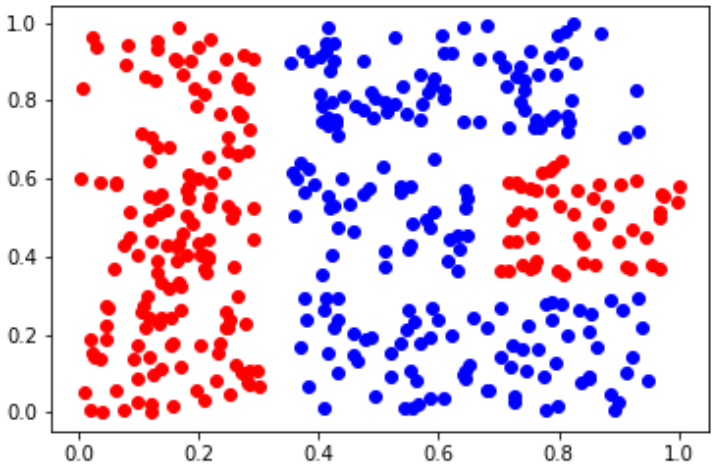
Fitting a kernel SVM
(polynomial kernel, degree 2)



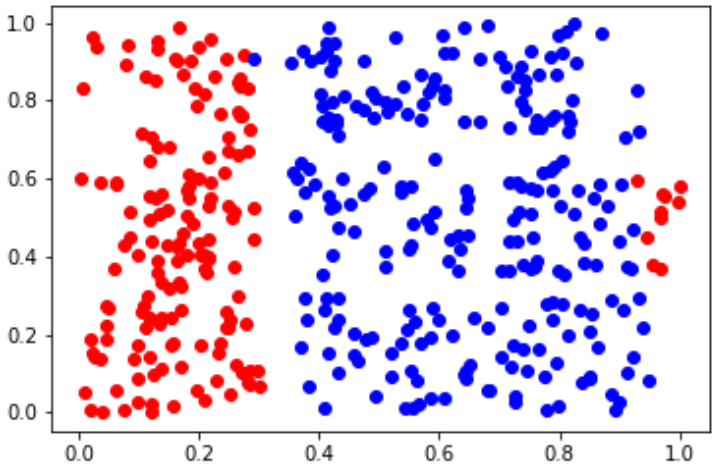
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



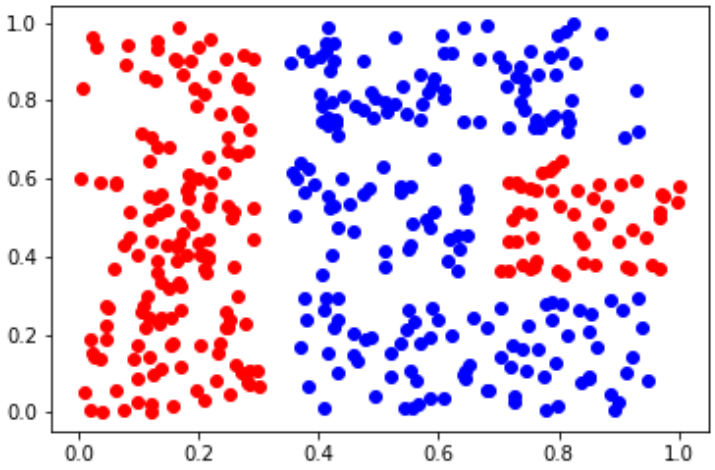
Fitting a kernel SVM
(polynomial kernel, degree 4)



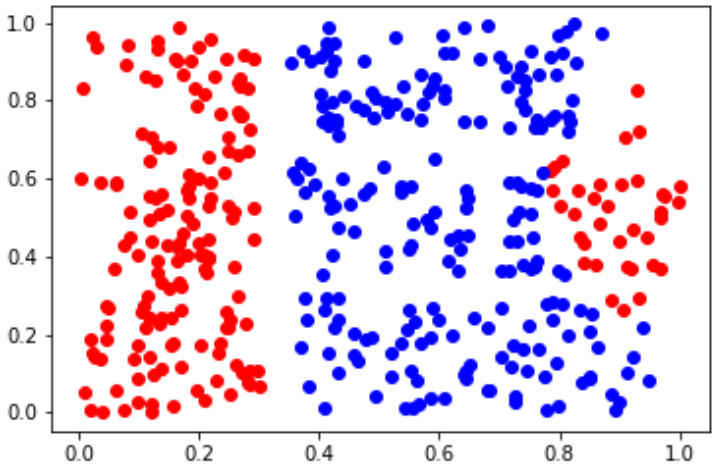
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



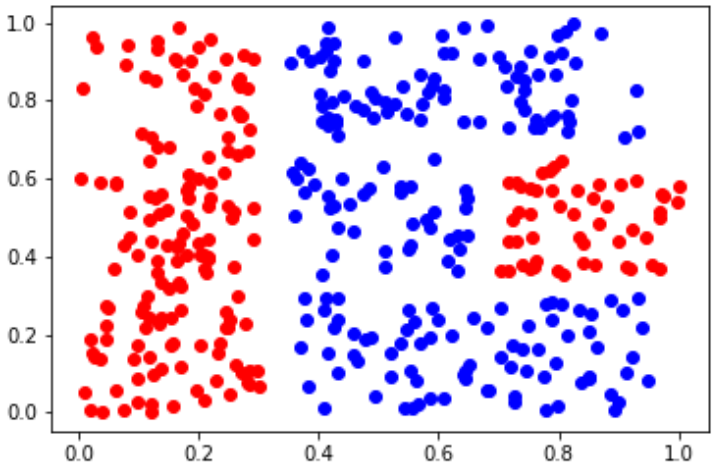
Fitting a kernel SVM
(polynomial kernel, degree 8)



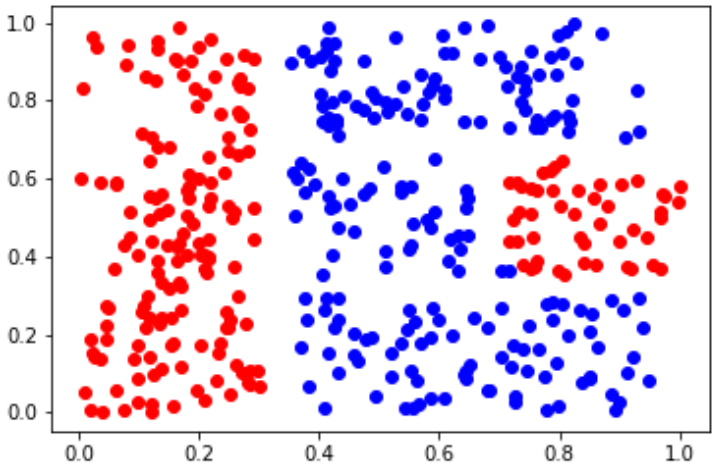
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



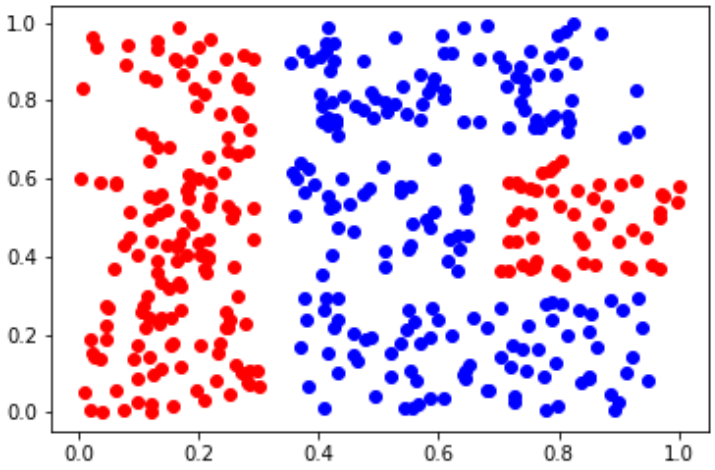
Fitting a kernel SVM
(polynomial kernel, degree 16)



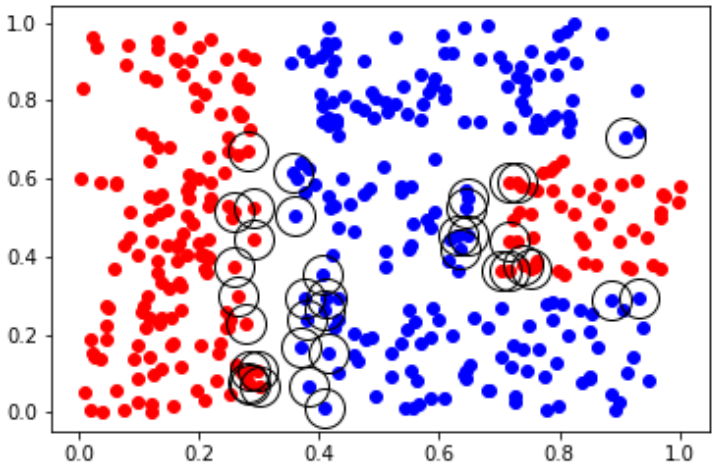
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



Fitting a kernel SVM
(polynomial kernel, degree 16)

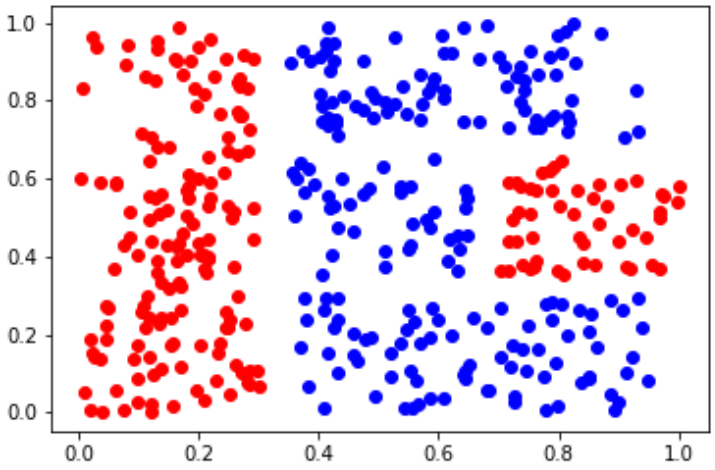


Circles indicate support vectors

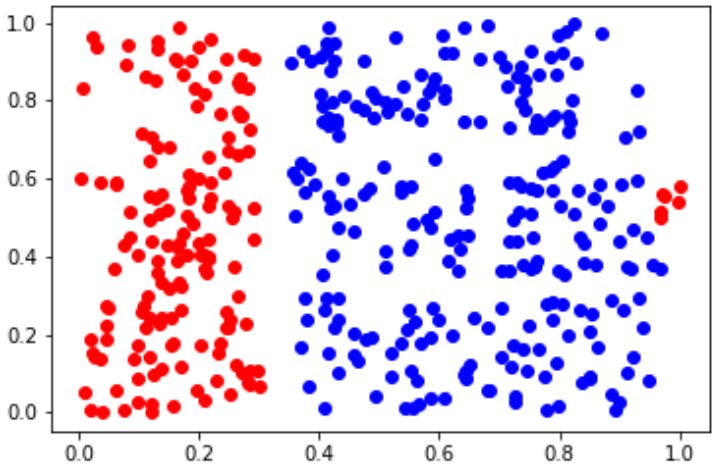
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



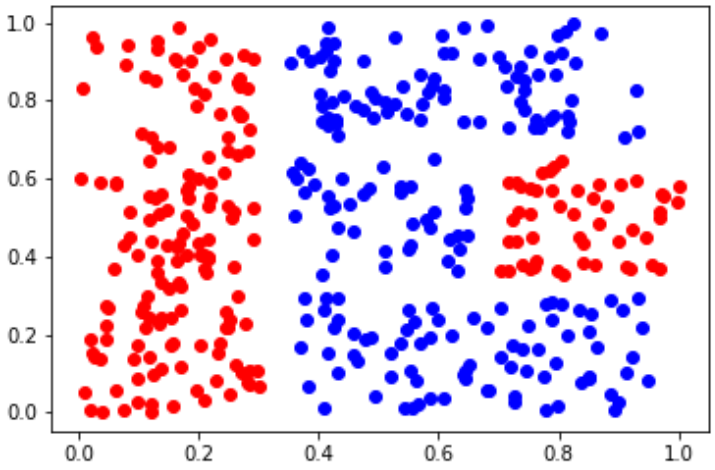
Fitting a kernel SVM
(RBF kernel, $\sigma^2 = 1$)



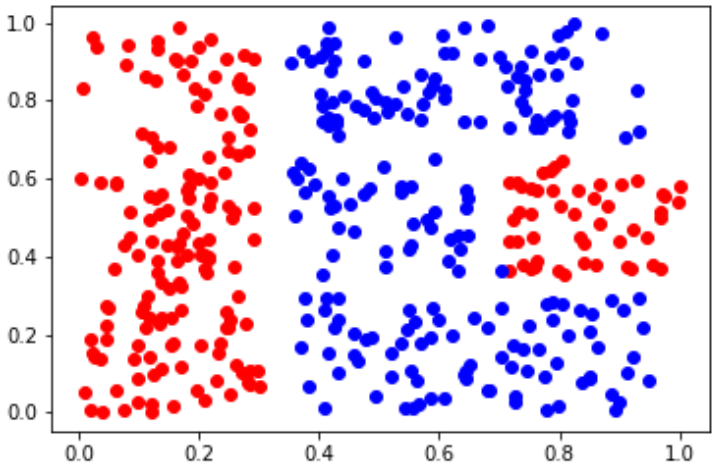
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



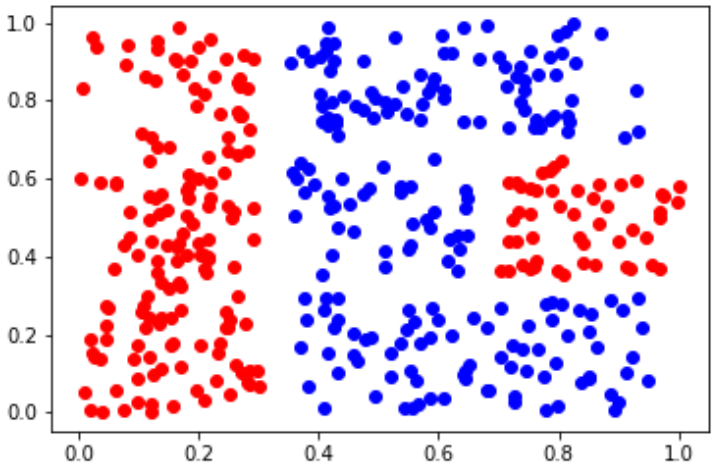
Fitting a kernel SVM
(RBF kernel, $\sigma^2 = 10$)



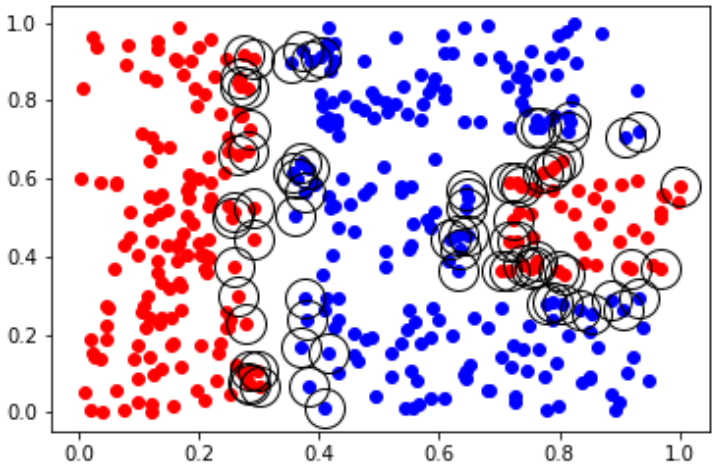
Kernel SVM: Example 2

- 2D inputs from 2 classes (colors)

True labels



Fitting a kernel SVM
(RBF kernel, $\sigma^2 = 10$)



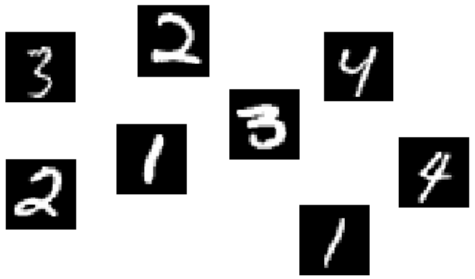
Circles indicate support vectors

Kernel SVM: Demo

- https://jgreitemann.github.io/svm-demo

Kernel SVM on MNIST

- Digit classification



Method	Preprocessing	Error	Reference
K-NN with non-linear deformation (P2DHMDM)	shiftable edges	0.52	Keysers et al. IEEE PAMI 2007
SVMs			
SVM, Gaussian Kernel	none	1.4	
SVM deg 4 polynomial	deskewing	1.1	LeCun et al. 1998
Reduced Set SVM deg 5 polynomial	deskewing	1.0	LeCun et al. 1998
Virtual SVM deg-9 poly [distortions]	none	0.8	LeCun et al. 1998
Virtual SVM, deg-9 poly, 1-pixel jittered	none	0.68	DeCoste and Scholkopf, MLJ 2002
Virtual SVM, deg-9 poly, 1-pixel jittered	deskewing	0.68	DeCoste and Scholkopf, MLJ 2002
Virtual SVM, deg-9 poly, 2-pixel jittered	deskewing	0.56	DeCoste and Scholkopf, MLJ 2002

Discussion

- Discuss two advantages and two drawbacks of kernel methods